

Primitive Recursive Functions

Name: Abraham Ladha, John Hill

1.1 Primitive recursion

Long ago, we thought the chemical elements to be distinct, but we now know them as just permutations of the same fundamental building blocks; atoms. In the same way, you may notice that all functions which occur to you naturally are made up of smaller, more atomic functions, combined via composition. Take a moment to think of a set of small atomic functions which could not be made up of smaller functions.

These three functions we define to be primitive recursive:

$$\text{Zero: } N(x) = 0$$

$$\text{Successor: } S(x) = x + 1$$

$$\text{Projection: } U_i^n(x_1, \dots, x_n) = x_i$$

A function is primitive recursive if it is obtained by applying two operations to other primitive recursive functions. Those two operations:

- Composition: If $g_1, \dots, g_n, f \in PR$, then $f(g_1(\cdot), \dots, g_n(\cdot)) \in PR$.
- Primitive Recursion: For $f, g \in PR$, we say h is PR when $h(0, x_1, \dots, x_k) = f(x_1, \dots, x_k)$ and $h(S(y), x_1, \dots, x_k) = g(y, h(y, x_1, \dots, x_k), x_1, \dots, x_k)$.

Definition 1.1 *The set of primitive recursive functions (PR) is the smallest set containing N, S, U , and maintains closure under composition and primitive recursion.*

The composition operation is quite natural. For the primitive recursion operation, the first parameter of h is the depth of recursion. The first statement says that the base case is PR . The second statement is recursive. This is saying that $h(y, \dots)$ being PR will imply that $h(S(y), \dots)$ is PR .

To prove a function $h(y, x)$ is primitive recursive is similar to induction, and requires two steps. First show $h(0, x)$ is primitive recursive. Then assuming $h(y, x)$ is primitive recursive, show $h(S(y), x)$ is also primitive recursive. This is not so intuitive, and I believe is better learned by example:

Here we show that addition is primitive recursive.

- $add(0, x) = U_1^1(x)$
- $add(S(y), x) = S(add(y, x))$

Once you prove a function is primitive recursive, then you may use it to build up other more complex functions, and show those are primitive recursive. Here we will show multiplication, built from addition.

- $mul(0, x) = N(x)$
- $mul(S(y), x) = add(mul(y, x), x)$

1.2 PR $\subsetneq$ R

Every primitive recursive function is computable¹ but the converse is false. Not every computable function is primitive recursive. One example is the Ackermann function, constructed exactly to be not so:

$$A(m, n) = \begin{cases} n + 1 & m = 0 \\ A(m - 1, 1) & m > 0, n = 0 \\ A(m - 1, A(m, (n - 1))) & m, n > 0 \end{cases}$$

It grows really fast. $A(4, 3) = 2^{2^{2^{2^2}}} - 3$. This number does not fit into any calculator I have tried. The primitive recursive functions correspond to turing machines whose time complexity is bounded in the size of the input. If you think of code, any for loop must have the number of iterations fixed before the loop starts.

I don't have any particular feelings about the Ackermann function². Here is another kind of proof of existence of a computable function which is not primitive recursive. We proceed by diagonalization. Let $\psi_1, \psi_2, \dots$ be an ordering³ of primitive recursive functions. Then let $\phi(x, y) = \psi_x(y)$. That is ϕ takes on two integral arguments, and is equal to the x 'th primitive recursive function of variable y . Let $\psi(x) = \phi(x, x) + 1$. Assume to the contrary $\psi(x)$ is primitive recursive. Then there is index n for it. There exists n such that $\psi(x) = \psi_n(x)$. Then by definition, $\psi(x) = \phi(n, x)$. Take $x = n$. It then follows that

$$\phi(n, n) = \psi_n(n) = \psi(n) = \phi(n, n) + 1 \quad (1.1)$$

A contradiction! Therefore, ψ is not primitive recursive.

¹Exercise, see 9.

²You may find a clean proof that $A \notin PR$ here: <http://www.cs.tau.ac.il/~nachumd/term/42019.pdf>

³Convince yourself that PR is a countable set, so there must exist some bijection $\mathbb{N} \rightarrow PR$. In the next sheets we will learn about Gödel numberings. This ordering is a weaker version of that.

1.3 Problems

Turn in 6 of the first 8 problems along with 9 and 10. You may use the results of problems in your solutions to other problems.

1. Let $P(0) = P(1) = 0$ and $P(S(x)) = x$ otherwise. Prove that P is primitive recursive.

Proof: First we have $P(0) = N(0) = 0$ and similarly $P(1) = N(1) = 0$. Now assume that $P(x)$ is primitive recursive, now we need to show that so is $P(S(x))$, so we have $P(S(x)) = x$, but note that $P(S(x-1)) = P(x) = x-1$ so $P(S(x)) = x = S(P(x))$. Hence P is a primitive recursive function. ■

2. Prove that the identity function, $id(x) = x$ is primitive recursive.

Proof: We have $id(x) = x$. Now first note that $id(0) = 0 = N(0)$. Now assume that $id(x)$ is primitive recursive. And note we have $id(S(x)) = x+1$ and $S(id(x)) = x+1$ so we get $id(S(x)) = S(id(x))$ i.e a composition of primitive recursive function. Hence we show that id is a primitive recursive function. ■

3. Prove that for any constant c , $f(x) = c$ is primitive recursive

Proof: Note that as long as we can represent f is a composition of primitive recursive function we show that f is primitive recursive. So in this case for a fixed c simply consider $S \circ S \dots S \circ S(0)$, c times. Note that applying it once gives us $S(0) = 1$, applying it twice gives us $S(S(0)) = S(1) = 2$ and inductively applying it c times gives us c . So we have for ally x that, $f(x) = S \circ \dots \circ S(0)$ which makes f primitive recursive. ■

4. Prove $x!$ is primitive recursive

- $0! = 0 = N(0)$
- $S(x)! = mul(x!, S(x))$

5. Prove x^y is primitive recursive

- $exp(0, x) = 1 = S(0)$
- $exp(S(y), x) = mul(x, exp(y, x))$

6. Let $\multimap$ represent truncated subtraction. If $x - y$ is non-negative, then $x \multimap y = x - y$, and $x \multimap y = 0$ if $x < y$. Prove truncated subtraction is primitive recursive.

Proof: Consider $trunc(x, y) = x \multimap y$. Now fix a x and note,

- $trunc(x, 0) = x$
- $trunc(x, S(y)) = P(trunc(x, y))$

Note here that we define P as the predecessor so we have $P(0) = 0$ and $P(S(x)) = x$ so it's primitive recursive. ■

7. Prove that $\max(x, y)$ is primitive recursive.

8. Compute⁴ $A(1, 2)$ and $A(3, 3)$.

⁴It is okay to use programming, just mention how you did it.

9. Prove every primitive recursive function is computable. (Hint: use induction on the depth of recursion. The base case is one of N, S, U)

Proof: We do induction on the depth of recursion to claim that for any primitive recursive function f with depth n , f is computable. Start with the base case, with $n = 0$, so we have the three primitive recursive functions N, S, U_i^n . Note that these are trivially computable, this is because we can easily compute the value using an algorithm (for N just return 0, for S on $x \in N$ return $x + 1$ and for U_i^n just return the i 'th element). So we showed the base case. Now assume true for arbitrary depth $\leq k$. Now consider a function f built using $k + 1$ steps, so either it was built using composition of functions with depth $\leq k$ or it was built using primitive recursion from those functions. Now in the first case let $f = h(g_1, \dots, g_i)$ where each $h, g_1, \dots, g_i$ are computable functions with depth smaller than equal to k . Now note that first for a given input x we can compute $g_1(x) \dots g_i(x)$ because of our inductive hypothesis say they return $x_1 \dots x_i$, so we have now $f(x) = h(x_1, \dots, x_i)$ but now note that h is also computable because of our assumption, hence we have an algorithm to solve $h(x_1, \dots, x_i)$ to get $f(x)$. So we have a finite number of computable steps to figure out $f(x)$ and hence f is computable. Now in the second case i.e primitive recursion, let f be defined as,

$$\begin{aligned} f(0, x_1, \dots, x_k) &= h(x_1, \dots, x_k) \\ f(S(y), x_1, \dots, x_k) &= g(y, f(y, x_1, \dots, x_k), x_1, \dots, x_k) \end{aligned}$$

Now clearly for $f(0, x_1, \dots, x_k)$ we simply compute the value of $h(x_1, \dots, x_k)$ which we know is computable because h is a computable function as it's depth is smaller than equal to k . Now for the other case, we need to compute $f(y, x_1, \dots, x_k)$ to get $f(S(y), x_1, \dots, x_k)$. So what we can do is we start from $f(0, x_1, \dots, x_k)$ and then because g is computable we can compute $f(S(0), x_1, \dots, x_k)$ as that is equal to $g(y, f(0, x_1, \dots, x_k), x_1, \dots, x_k)$ all of which are computable. We do this recursively y times to get the value for $f(S(y), x_1, \dots, x_k)$. As we do it in a finite number of steps and as each step is computable because the functions involved i.e g are computable, the entire process is computable.

So we've showed using induction that all primitive recursive functions are computable. ■

10. Let the Collatz function be

$$C(n) = \begin{cases} n/2 & \text{if even} \\ 3n + 1 & \text{if odd} \end{cases} \quad (1.2)$$

Let $D(n)$ be the program such that $x = n$, while $x \neq 1, x \leftarrow C(x)$. Prove that if D is primitive recursive, then the Collatz conjecture is true.

Proof: If D is primitive recursive then by 9. we know that it's computable (so we must be able to find the solution in a finite number of steps as we showed above). Hence there is a finite algorithm that will eventually halt on any input x . But for this to be the case eventually we must have $x = 1$ i.e for any x the sequence of $(x, C(x), C(C(x)), \dots)$ must reach 1 in some finite number of steps. This implies that the Collatz conjecture is true. ■